bookwriter — Installation and User Manual

Installation and user guide

Version 0.1.0

bookwriter turns your own texts into a book. It reads what you have already written, cuts it into sections, understands what each section is about, groups the sections into chapters, and produces a finished document — either in your exact words or rewritten in a style you choose. It can also rewrite an existing book, translate it, or update a draft with material it has not used yet.

Everything can run on your own computer. Your text is sent only to the model service you configure, which by default is LM Studio running locally.

Part 1 — Installation

1.1 What you need

- **Windows** with PowerShell (the installation scripts are written for it). The program itself also runs on macOS and Linux, but you must then create the environment by hand — see §1.5.
- **Python 3.10 or newer**. Download from python.org and tick *Add Python to PATH* during setup.
- **A model service.** Either:
  - **LM Studio** (free, runs on your machine) — https://lmstudio.ai/, or
  - an account with a cloud provider (Nebius Token Factory is supported out of the box).
- **About 2 GB of free disk space** for the Python dependencies, plus whatever the models take.
- **A GPU with 8 GB of video memory or more** is strongly recommended if you run models locally. Everything works on the processor alone, but a book takes many hours instead of one.

1.2 Installing the program

Unpack the archive you received. Open PowerShell in the extracted folder (in File Explorer: right-click on empty space → *Open in Terminal*) and run:

```
.\install.ps1
```

The script finds your Python installation, creates a private environment in a `.venv` subfolder, downloads the dependencies and runs a check against LM Studio. The first run downloads roughly 1.5 GB and takes a few minutes.

Useful variants:

```
.\install.ps1 -Recreate # start over from a clean environment
.\install.ps1 -Extras "gui" # graphical interface only, no PDF/EPUB import
.\install.ps1 -SkipDoctor # do not check LM Studio at the end
```

Nothing is installed system-wide: everything lives inside the program folder. To uninstall, delete the folder.

1.3 Installing the models (LM Studio)

Open LM Studio and download two models. In LM Studio's search tab:

Role	What to look for	Suggestion
Writing	any *instruct* model that supports structured output	a 7B–35B instruct model; larger writes better and slower
Comparing meanings	an *embedding* model	`Qwen3-Embedding-0.6B` (multilingual, 640 MB)

The embedding model matters more than people expect. It decides which sections end up in which chapter. If your book is **not in English**, choose a multilingual embedding model: `Qwen3-Embedding-0.6B`, `bge-m3` or `multilingual-e5-large`. An English-only model on a non-English corpus produces nearly random chapters.

Then start the server: in LM Studio go to the **Developer** tab and switch the server on. It listens on `http://localhost:1234` by default, which is what bookwriter expects.

> **Tip on video memory.** LM Studio reserves memory for the whole context window multiplied by the number of parallel slots. A model loaded with a 262,000-token context can fill your card for nothing: bookwriter never needs more than about 32,000. If your system slows to a crawl, reload the model with a smaller context.

1.4 Starting the program

```
.\run.ps1
```

The desktop window opens. To use the command line instead, pass any command to the same script:

```
.\run.ps1 doctor # check the connection and the models
.\run.ps1 project list
```

1.5 Manual installation (any operating system)

```
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\Activate.ps1
pip install -e ".[all,dev]"
python -m bookwriter gui
```

1.6 Where your work is stored

Everything you produce lives **outside** the program folder, in a workspace directory:

```
~/.bookwriter/
■■ config.toml global settings (provider, models, interface language)
■■ registry.db the list of projects
■■ projects/<name>/
■■ project.db your texts, sections, themes, structure, token usage
■■ vectors.lance/ the semantic index
■■ config.toml settings for this project
■■ exports/ produced documents
```

Because the workspace is separate, replacing the program folder with a newer version never touches your books. To move the workspace elsewhere, set the environment variable `BOOKWRITER_HOME` before starting:

```
$env:BOOKWRITER_HOME = "D:\Books\bookwriter"
.\run.ps1
```

Part 2 — First run

2.1 Create a project

At the top of the window there is the project bar. Press **New…**, give the project a name, and optionally a target length in words (0 means "not set"). Everything you do afterwards belongs to the selected project; you can keep as many projects as you like and switch between them at any time.

At the far right of the same bar there is the **Language** menu: it changes the language of the interface only, not of your book. All twelve languages listed there are fully translated; the catalogues are plain JSON files in `bookwriter/gui/locales/` and can be corrected without touching the code.

2.2 Configure the models

Open the **Settings** tab and set, in this order:

1. **Provider** — *LM Studio* (on your computer, free) or *Nebius Token Factory* (cloud, paid). For a cloud provider an **Access key** field appears; the key is stored in your project configuration file on your machine. You can instead set the environment variable `NEBIUS_API_KEY`, which takes precedence.
2. **Project** — the settings below apply to the project selected here.
3. **Language of generated content** — the language of everything the model writes: section summaries, themes, chapter titles, prose. Your imported text is never changed. Leave it on *as in the source text* only if you are happy for the model to follow the language of each fragment.
4. Press **Read the available models**, then choose the **model that writes the texts** and the **model that compares meanings**. Models already loaded in memory are marked `(loaded)` in red and start immediately; the others are loaded on first use, which makes the first operation slower. A cloud provider also shows the price per million tokens next to each model.
5. **Model for the quality pass** is optional: a slower, more accurate model used only for style checks and consistency passes.
6. Press **Save the settings**.

The search prefixes required by each embedding family are applied automatically when you pick the model. Changing the embedding model later is safe: the index is detected as stale and rebuilt.

Part 3 — Making a book from your own texts

The three numbered tabs are the main path, in order.

3.1 Step 1 — Import texts

Drag your files onto the list, or use **Add files…** / **Add folder…**. Supported formats: `txt`, `md`, `pdf`, `docx`, `epub`. Tick *Include subfolders* to walk a whole directory tree. Press **Import the selected texts**.

The files are read and split into blocks of about 1,500 tokens. Their **structure** is recorded too — headings, chapters, page breaks, bold and italic — so that a book can later be handed back looking as it came in (§5.1). The original files stay where they are: only their text is copied into the project. Importing the same file twice is harmless — it is recognised by content and skipped.

Texts imported here are the project's **sources**: they stay. A book opened from *Rewrite* or *Translate* is working material instead, and does not survive a restart (§4.2) — unless it is still waiting in the queue, in which case it is kept until its job is done (§4.4).

3.2 Step 2 — Prepare content

Four buttons, to be used in this order.

Split into sections

This is the heart of the process. Your text is cut into **sections** at paragraph boundaries — never inside a paragraph — and each section is described by the model with a title, a one-line summary, themes, and the people and places it mentions.

You are asked for the indicative length of a section (default 400 words). What comes out is typically a section of 300–500 words: a complete argument that can stand on its own and be moved into a chapter without losing sense.

Two properties follow from cutting deterministically rather than asking the model to produce the text: **the whole text is covered**, and **the text is yours, word for word**. The model never returns your prose, only labels for it.

For 300 sections expect roughly half an hour on a local mid-size model. The progress bar shows the percentage and the count; the list at the bottom of the panel fills up as the work proceeds, so you can read the first results without waiting for the end. If you stop the program, the work already done is kept and the next run resumes.

Make searchable

Computes the semantic index of the text blocks. Needed for the search box at the bottom of the panel. Fast: a couple of minutes for a few hundred blocks.

Connect the ideas

Computes the semantic index **of the sections** and builds the thematic network that links related ones. Required before grouping into chapters.

Export browsable archive

Produces an HTML archive with one page per section, the themes, and a graph of the connections. Useful to explore what you have before deciding the shape of the book.

Searching

The box at the bottom searches **by meaning**, not by exact words: describe what you are looking for in your own words and choose whether to search *in the texts* or *in the ideas*.

3.3 Step 3 — Compose the book

① *Group the ideas into chapters*

Choose the ordering:

Ordering	What it does
by theme	groups sections that talk about the same thing (the usual choice)
in time order	uses the dates found in the text; useless if your text has no dates
like a story	asks the model for a narrative arc
by my own criterion	you describe the criterion in plain words

Set the number of chapters explicitly. Left at 0 the program picks at most 10, which for a large corpus means chapters of twenty thousand words. For 400 sections, 20–25 chapters is a sensible range. Inside each chapter the sections keep their original reading order.

② *Generate the book*

Two ways, and the difference matters:

Export anthology (original texts) — writes your own sections, reordered by chapter, with no rewriting at all. Instant, free, and your voice is untouched. For memoir and non-fiction built from your own writing this is often the best result.

Write the book in prose — the model writes new prose from your material. Three settings govern it:

- **Style** — a style profile (see §4.1), or *plain prose*.
- **length (words)** — the target for the whole book. The budget is divided among chapters in proportion to how much material each one holds.
- **Rendering** — the important one:
  - **faithful to your words** (default): the model composes rather than rewrites. It orders your passages, joins them with short transitions, merges those that say the same thing, removes repetitions and turns lists into prose. When the target is well below the material available, it is told exactly how much to keep and instructed to reduce *by subtraction* — first cutting repetitions, then merging, then trimming marginal detail, and only as a last resort shortening a passage by removing words rather than replacing them.
  - **free rewriting in the style**: the model rewrites everything in the chosen style. Use it when you deliberately want a different voice.

A realistic note on length: if you have 185,000 words of material and ask for 80,000, more than half of what you wrote will not appear. That is a decision, not a malfunction. If you want everything, set the target close to the material you have.

The models systematically write less than asked; bookwriter compensates by re-generating a chapter up to three times, keeping the attempt closest to the target.

Unify names and terms

A pass over the finished book that makes proper names and terminology consistent throughout. Run it after the prose is written.

3.4 Structure (edit)

The table of contents, editable by hand. Drag to reorder chapters and move sections between them, double-click to rename a chapter. Then:

- **Regenerate the titles** — replaces the mechanical labels (`theme / theme`) with real chapter titles written by the model, without touching how the sections are grouped.
- **Book title…** — the title printed at the top of every exported document.
- **Split chapters into sections** — adds a level of subheadings inside long chapters.
- **Save the structure** — required after manual edits.

Look at the structure before generating anything: correcting it here costs minutes, correcting a finished book costs hours.

Part 4 — The other modes

4.1 Writing style

A style profile describes writing along nine dimensions — syntax, punctuation, vocabulary, figures of speech, rhythm, register and so on — and is passed to the model whenever it writes.

Four ways to create one:

- **Learn from a text…** — point it at a document and it measures the style of that document. **To keep your own voice, use one of your own texts here**: this is the single most effective setting for fidelity.
- **Describe in words…** — write what you want in plain language.
- **Blend authors…** — one line per author, `Author:weight:dimension1,dimension2`, for example `Calvino:0.7:syntax,rhythm`.
- **Import from the library…** — reuse a profile saved in the global library, shared by all projects.

Select a profile and paste some text in the box below to see what it does before committing to it.

4.2 Rewrite in another style

Takes a whole document and rewrites it in the chosen style, keeping names and terminology consistent from beginning to end.

- **Open a file…** takes a document from your computer and selects it immediately — you are not limited to the texts already imported. The book you open here is **working material**: it is available for the whole session but it is not added to the project's sources, and it is gone when you reopen the program. To keep a text in the project, import it from *Import texts* instead.
- **Length (opt.)** is a ratio to the original: `1` the same, `0.7` about 30% shorter.
- **Check the style** (on by default) asks the model after each block whether the result respects the style and redoes it if not. More faithful, roughly two and a half times slower. Turning it off halves the work.
- **Format** — DOCX, PDF, Markdown or EPUB. See §5.1: the layout of the original is kept in the first three; an ebook has no pages to keep.
- **Preview (first chapter)** runs on the first chapter only, so you can judge before committing.

The work is resumable within the session: if a run is interrupted, the blocks already rewritten are kept. Reopening the program clears working documents, and with them any unfinished run on those documents.

4.3 Translate

The same engine with a target language. It carries a bilingual glossary from block to block, so a character or a technical term is not translated three different ways.

Type the language in plain words (`English`, `en`, `inglese` all work), optionally choose a style, pick the output format, and use the preview first. This is deliberately sequential — that is what keeps the

terminology consistent — so expect around twenty minutes for 25,000 words on a local model with style checking on.

Only the text is translated: headings, chapters, page breaks, blank pages and character formatting come back exactly as they were (§5.1). As in *Rewrite*, a book opened here is working material and does not join the project's sources.

4.4 Work queue

Translating or rewriting a book takes hours. Staying at the computer to launch the next one when the first finishes is a person's time spent doing a counter's job.

In *Rewrite* and *Translate*, **Add to the queue** puts the job in line instead of starting it: you choose the output file's **name and folder** right there, and the book waits its turn. You stay on the panel you were using and your document stays selected, so queueing five books in a row does not mean finding each one again in the list. The **Work queue** tab shows the line — book, operation, format, output file, state — and lets you reorder it with ↑ and ↓, change a destination, or drop an entry. **Start the queue** works through it from the top.

The queue is stored in the project, not in the window. Close the program and it is still there when you come back, with its order, its destinations and how far each book got. A working document (§4.2) whose job is still pending is kept for the same reason: it is swept away only once its book has been delivered or removed from the queue.

One at a time, deliberately: two books running together compete for the same model and both finish later than they would have in single file. **A book that fails does not stop the queue** — the reason is recorded in its row and the next one starts, because the point of leaving a queue running is to come back and find everything that could be done, done.

4.5 Interrupting

A long job can be stopped: the **Interrupt** button appears in the status bar next to the progress bar while a rewrite, a translation or the queue is running.

It stops at the end of the passage being processed, never inside one. Everything already produced is saved as it goes, so nothing is lost: restarting the same operation picks up exactly where it left off, and the model is not asked twice for the same passage. **No file is written for an interrupted book** — a half book that presents itself as finished is worse than no book.

The queue also has its own gentler stop, **Stop after the current book**: the book in hand is taken to the end and its file written, and the ones after it are not started.

An interrupted book restarts from where it stopped: press **Start the queue** again, or use «Translate the whole book» on the same document and language. If that book was queued, the document it came from is kept across restarts too (§4.4), so the interruption survives closing the program.

4.6 Write from scratch

For a book you have not written yet. You talk the book through with the assistant, and then it makes it, in seven phases. The strip of buttons at the top of the tab shows where you are; click one to move around, or use **Move to the next phase**, which tells you what is missing before it lets you past.

1 · Interview. It is a conversation, not a questionnaire. Tell the assistant about the book the way you would tell an editor — an idea barely sketched is enough — and it works like one: it asks one thing at a time, tied to what you just said; it stretches the plot by asking what happens next and why; it points out the holes (an event with no cause, a character acting without a motive, an ending that contradicts the message); it proposes, and waits for your yes before treating anything as decided. Its territory is what you want the book to achieve and the effect it should have on the reader; who that reader is, and what follows for language, register and chapter length; the chain of events and the logical steps; the psychological mechanisms and the way facts connect; the scenes and passages you want, and the ending; the characters — how they look, how they think, their temperament and culture — and the places. It does **not** ask about printing, margins, cover, binding or channel, which you decide by hand in production; and it does not ask which style to use, because the style is the one chosen in the **Style** drop-down at the top right of the tab (it knows it, and at most suggests a refinement).

Write in the box at the bottom: Enter sends, Shift+Enter starts a new line. The reply appears as it is written, and until it starts a line says how long the model has been thinking; **Stop** interrupts it, keeping what has already arrived. What you write while the assistant is busy is queued and sent as soon as it is free. If a reply does not arrive, what you wrote is saved and **Try again** sends it again.

Beside the conversation, the **book notebook** grows after every exchange: plot, chain of events, how facts connect, characters, places, what the book wants to achieve, what was agreed together and what is still open. It is where you notice a misunderstanding while fixing it costs a sentence; the **Book notebook** button shows or hides it.

You decide when the conversation is over. **I'm done: you decide the rest** closes it: the assistant completes the notebook, settles every open question and tells you, one line each, what it decided on its own. The conversation stays open — write again and the notebook follows. The notebook is what the following phases work from: the structure is designed on the chain of events and the key scenes, and each chapter is written knowing who the characters are and where the book is going.

2 · Specification. Propose the structure and write the specification produces `MASTER_SPEC.md`: the notebook in order — the book, the reader, the plot, key scenes and ending, characters, the world — the chosen style, and a table with one line per chapter and its word budget. Read it, edit it if you like — by hand, with **Edit the specification…**, or in any text editor — and then **Approve**. Revising an outline costs minutes; revising twenty written chapters costs days.

3 · Research. See what already exists looks up the books that are already out on the subject, their tables of contents and their recurring criticisms, and writes `research/00_landscape.md`, whose consequences change the structure — which is why it comes before the structure is fixed. **Find the chapters' sources** then gathers what each chapter needs, one file per chapter, tagging every claim with its level of evidence. Nothing without a source reaches the manuscript: a figure that cannot be traced back stays in the research file, under «questions still open». If you would rather not, **Write without research** says once what that costs and moves on.

This is the only part of the program that goes out to the internet, and it ships switched off. Turn it on in **Settings** → **Search the web too** and pick an engine: DuckDuckGo asks for nothing, Brave and Tavily want a key, SearXNG wants the address of your own instance. Left off, research works on the texts you imported into the project.

4 · Writing. Write the next chapter, or **Write all the chapters** with the progress bar and the stop button. Each chapter is written to `<project>/book/chapters/NN_name.md` the moment it is done, before the next one starts — a book spans many working sessions, and what is not on disk when one

ends is not recoverable by anyone. That is also why picking up again restarts from the first missing *file*, not from where anyone believes they got to. The same row writes the front and back matter: introduction, preface, notice, glossary, author biography, acknowledgements.

A chapter the model fails on does not take the others down with it: the run carries on and the report names it at the end, with the reason, under **NON RIUSCITI**. Nothing was written for it, so pressing the button again simply picks it up along with whatever else is missing. Three failures in a row, though, stop the run — at that point it is not the chapter, it is the model or the server, and trying the remaining twenty would mean waiting out a full timeout on each to be told the same thing twenty times.

5 · Production. Choose the cover artwork…: the cover comes from you, and is never invented nor generated unless you ask. The panel says whether the image holds up at the chosen size, and if it is too small it gives you the number — 1819×2551 pixels for A5 with bleed — instead of quietly enlarging it. Pick the **Page size**, then **Write book.yaml**, which collects into one file the title and language from the notebook and the object's decisions — size, imprint, author as printed, rights — from the project's production settings.

6 · Composition. Tick the formats and press **Compose and export**. Out come a print PDF at the exact trim with the cover as page one, the interior on its own (which is what a printer and KDP actually want), the cover files, a DOCX with real styles and a live contents page, and an EPUB 3. Without artwork the build stops and asks for it; **even without a cover** composes the interior as a proof.

7 · Checks and handover. The print checks run by themselves after every build: page count parity, page size against the declared trim, fonts not embedded, text too close to the trim, chapters landing on a left-hand page, EPUB structure, DOCX fields. **Write the production notes** puts in `output/PRODUCTION_NOTES.md` what is still open — ISBN, font licensing for a commercial run, spine width, back cover copy — and says plainly what these checks do not cover: they replace neither Kindle Previewer for the ebook nor a printed proof for the paper edition.

State on disk, at the bottom, is available in every phase. It looks at the files that really exist against those the specification plans, and names the chapters that are not there. It is the only report worth trusting.

Putting a book aside, and starting over. The seven phases write to disk at every step, which is why the work is never lost — and also why closing the program does not end it: reopening finds the book exactly where it was. The bottom row of the tab is how you leave it behind.

Save the work puts the book aside as it stands, in a single zip file inside `<project>/saves/`, and the work carries on from where you are — it is a copy, not an exit. Give it a name you will recognise later; the list shows the date, the phase, how many chapters and how much it weighs.

Resume a saved book… puts one back into production. What is in the folder at that moment is saved first, by itself, so returning to an earlier point never costs you the later one.

Start over clears the seven phases and goes back to the interview. It first says what is in the current book — chapters, words, phase — and asks what becomes of it: *Save and start over*, which is the preselected answer and leaves it in the list of saves, or *Delete and start over*, which is permanent. If the save fails for any reason, nothing is deleted: you asked for it to be kept. Imported documents, the project database and everything outside `book/` are never touched.

The same three actions exist at the command line: `forge save`, `forge saves`, `forge restore`, and `forge reset` (which puts the book aside unless you pass `--delete`).

4.7 Complete the content

The assistant analyses what the book is missing and proposes questions. You answer in your own words, press **Add the answer to the book**, and your answer becomes new material, ready to be organized and written like everything else.

4.8 Update a book

For when you already have a draft and new material has arrived.

Load the draft with **Load the draft…**, then press **Compare and update**. The program reads the draft's structure, compares every paragraph against every section of the project by meaning, and produces the whole book again with four kinds of change marked:

Mark	Meaning
framed in red	new content, present in the project but missing from the draft
struck through in red	a paragraph that repeats something already said elsewhere
struck red then green	a paragraph that was incomplete, merged with the fuller material
plain	unchanged

Material that does not belong to any existing chapter is collected in a chapter named *New content to place*.

Two files are written: the book in the **format you chose** — DOCX (colours, strikethrough and frames are real Word formatting), PDF or Markdown, all three carrying the same marks — and an **HTML** version, for reading quickly in a browser.

Merge incomplete passages can be turned off: the partial content is then appended separately instead of being merged, and no model call is made.

4.9 Project summary

Topic, decisions taken, content, open questions — a compact picture of where the project stands. Useful when you come back to a book after a break.

4.10 Model usage

How much work the models have done, broken down by project and by activity: number of runs, requests, tokens sent, tokens generated, and — with a paid provider — the cost, computed with the prices in force at the time and frozen into the record.

The count updates every ten seconds while an operation runs, so this tab doubles as a sign of life during long jobs, and it survives closing the program.

Part 5 — Files and naming

5.1 Output format and layout

Every screen that produces a book has a **Format** menu: **DOCX**, **PDF**, **Markdown** or **EPUB**. The format is your choice, not a consequence of what you started from — a PDF can come back as a DOCX, a DOCX as a PDF.

All of them come out of the same typesetter as the seven phases (§4.6), so a book exported from here is set like a book and not printed like a document: real trim size instead of A4, asymmetric margins that account for the fold, hyphenation, chapters opening on a right-hand page, running heads, a contents page, and the per-chapter sources merged into one bibliography at the back.

EPUB is the exception to what follows, and not out of laziness: an ebook has no pages to preserve — the typeface and the size are chosen by whoever reads it. That is also why *Update a book*, whose file exists to be read for its marked changes, does not offer it.

When a book is translated or rewritten, **only the text is sent to the model**. Everything else is the program's business and is put back where it was:

Kept	How
headings and chapters	recognised at import, with their level, and rewritten as headings
page breaks and blank pages	never sent to the model; reinserted at the same points
bold and italic	carried as `**bold**` / `*italic*` and rebuilt on the corresponding words
tables	translated cell by cell, the table itself untouched
paragraph divisions	the model is asked for the same paragraphs, in the same order

If you start from a DOCX and ask for a DOCX, the original file is reopened and the text replaced inside it. Fonts, sizes, colours, margins, headers, footers, images and styles are therefore not reconstructed — they are never touched at all. Keep the source file where it was: if it has been moved or deleted, the program falls back to building a new document from the structure it recorded at import, which keeps headings, chapters and breaks but not the typography.

Two limits, stated plainly:

- A paragraph written in several different fonts is unified per emphasis level (the italic keeps the italic's own size and colour, the plain text the plain text's). Text of a different length cannot be poured into the same runs, so the runs are rebuilt.
- A PDF has no structure of its own: it is a drawing of characters. Headings are recognised by type size and weight, paragraph ends by line width. It works well on a typeset book, but it is inference, not a declaration — unlike DOCX and EPUB, which state their structure.

Books imported before this existed have no recorded structure. They still export, in the old plain rendering, and the program says so. Re-import them to get the layout back.

5.2 File names

Every generated document is saved through the standard Windows save dialog, with a suggested name that describes what it contains:

```
myproject_faithful_MyVoice_EN_80k.docx
■ ■ ■ ■ ■ target length
■ ■ ■ ■■■ language of the content
■ ■ ■■■■■■■■■■■■ style profile used
■ ■■■■■■■■■■■■■■■■■■■■■■ rendering mode (absent when free rewriting)
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ project or source document
```

Different styles, languages, lengths or modes therefore produce different files instead of overwriting each other. Translating a document replaces the language tag rather than adding a second one: `notes_prose_EN` becomes `notes_prose_IT`.

The folder you choose is remembered and proposed next time, including after restarting the program.

Part 6 — Troubleshooting

A button seems to do nothing. Look at the result line under the buttons: it reports what happened, including "nothing to do" cases. The counters line above it shows how much of each step is already complete.

The system slows down or freezes; memory spills into shared. The chat model is loaded with too large a context. Unload it and reload it with a smaller one:

```
lms unload <model>
lms load <model> -c 32768 --parallel 4 --ttl 1800
```

The `--ttl` argument unloads the model automatically after 30 minutes of inactivity.

"The model did not produce any text (reason: length…)". The token limit was consumed by the model's internal reasoning before it started writing. This should not happen with the shipped settings; if it does, the message tells you which model and how many tokens were involved.

Requests time out. Increase `request_timeout_s` in the project's `config.toml`. The default of 900 seconds is generous, but a very large model on a slow machine can exceed it.

A section was not processed. The result line reports how many failed and names the model. The usual cause is a model that is not loaded, or that does not support structured output. Check it in the Settings tab, then run the same step again: what is already done is skipped.

Chapters look repetitive or all alike. Check that a style profile is selected, and prefer *faithful to your words* unless you deliberately want a rewrite.

Chapters make no sense. Almost always the embedding model. On a non-English corpus, an English-only embedding model produces near-random groupings. See §1.3.

"An operation is already running." Long operations are deliberately not allowed to overlap: two writing runs at once would produce duplicate chapters. Wait for the first to end.

Starting over. Every step is idempotent: re-running it processes only what is missing. To start a step from scratch, delete the corresponding data — the safest way is to create a new project and import the texts again.

Appendix A — Command line

Everything the interface does is also available from the command line. Some examples:

```
.\run.ps1 doctor # check models and connection
.\run.ps1 project new "My book" --target-words 60000
.\run.ps1 ingest "D:\notes" -r # import a folder recursively
.\run.ps1 memory stats
.\run.ps1 memory search "what I am looking for"
.\run.ps1 organize --ordering thematic --chapters 20
.\run.ps1 compose --target-words 60000
.\run.ps1 export --format docx
.\run.ps1 rewrite --style "My voice"
.\run.ps1 translate --to English
.\run.ps1 jobs # state of long-running work
```

Add `--help` to any command for its options.

Appendix B — Configuration file

`~/.bookwriter/config.toml` holds the global defaults; each project has its own copy that overrides them. The settings worth knowing:

Setting	Meaning
`llm.provider`	`lmstudio` or `nebius`
`llm.base_url`	address of the model service
`llm.api_key`	key for a cloud provider (an environment variable takes precedence)
`llm.models.reasoning`	the model that writes
`llm.models.embedding`	the model that compares meanings
`llm.models.heavy`	optional model for quality passes
`llm.request_timeout_s`	seconds before a request is given up (default 900)
`llm.concurrency`	parallel requests for indexing and section labelling (default 4)
`chunking.target_tokens`	size of the text blocks (default 1500)
`project.language`	language of generated content
`ui.language`	language of the interface

Appendix C — Licence

The software is distributed under a proprietary licence: use is granted only to the person who personally downloaded the package. Redistribution and commercial use are not permitted. The full terms are in the `LICENSE` file.

Whatever you write with it is entirely yours. The licence places no restriction on the books you produce.